

Documentazione del progetto

1. Requisiti Informali

Il sistema da realizzare è una piattaforma di e-commerce, pensata per la vendita di prodotti online. L'idea di partenza è quella di un negozio digitale in cui un visitatore può registrarsi, autenticarsi, consultare un catalogo di prodotti, riempire un carrello e confermare un ordine, mentre chi gestisce la piattaforma tiene sotto controllo prodotti e ordini.

Distinguiamo tre tipi di attori che hanno a che fare col sistema. Il **visitatore** è chi non ha ancora fatto l'accesso: può solo registrarsi o autenticarsi. Il **cliente** è un utente registrato che compra: consulta il catalogo, gestisce il proprio carrello, conferma gli ordini e consulta lo storico dei propri acquisti. L'**amministratore** è chi gestisce la piattaforma: si occupa dei prodotti del catalogo e degli ordini di tutti i clienti.

Sul piano delle funzionalità, il sistema deve permettere a un visitatore di **registrarsi** fornendo i propri dati (nome, cognome, email e password, con la possibilità di indicare anche un indirizzo e un'immagine del profilo) e di **autenticarsi** con le proprie credenziali. L'email fa da identificatore, quindi non possono esistere due utenti con la stessa email.

Una volta autenticato, l'utente può **consultare il catalogo** dei prodotti e vedere per ognuno le informazioni principali (nome, descrizione, prezzo, quantità a magazzino, categoria, se è disponibile e se è in offerta). Il cliente può **aggiungere un prodotto al carrello** indicando una quantità: il sistema non deve mai permettere di mettere nel carrello più pezzi di quelli disponibili a magazzino, tenendo conto anche di quanto è già stato aggiunto in precedenza. Il cliente può poi **modificare il carrello**, cioè cambiare la quantità di una riga oppure rimuoverla del tutto portando la quantità a zero.

Quando il carrello è pronto, il cliente può **confermare l'ordine**. In quel momento il sistema verifica che tutte le quantità siano ancora disponibili, calcola il totale, crea l'ordine con la data e lo stato iniziale, scarica il magazzino delle quantità acquistate, svuota il carrello e invia al cliente una notifica di conferma. Il cliente può in seguito **consultare lo storico dei propri ordini**, mentre l'amministratore può **visualizzare l'elenco completo** di tutti gli ordini registrati nel sistema.

Oltre a queste funzionalità di base ne sono state previste altre a livello di analisi, che completano il quadro di un e-commerce reale: la gestione del profilo personale, la ricerca dei prodotti e la sezione offerte, la gestione del catalogo da parte dell'amministratore (aggiungere, modificare ed eliminare prodotti), l'aggiornamento dello stato di un ordine lungo il suo ciclo di vita, l'annullamento di un ordine e il monitoraggio della piattaforma tramite statistiche.

Ci sono infine alcuni requisiti che non riguardano una singola funzione ma il sistema nel suo insieme. Il sistema deve poter girare su più piattaforme (Windows, Linux, macOS, etc...). Deve

prevedere un meccanismo di notifiche per tenere informato il cliente sugli aggiornamenti dei suoi ordini. Deve rispettare degli standard minimi di sicurezza sui dati personali e sull'accesso, per esempio non memorizzando le password in chiaro.

2. Analisi e Specifica dei Requisiti

In questa sezione riportiamo gli artefatti prodotti durante l'analisi: i requisiti strutturati, il modello di dominio, gli attori e i casi d'uso con i relativi flussi degli eventi. Alla fine è riportata la tracciabilità che lega requisiti e casi d'uso.

2.1 Requisiti specificati

Partendo dai requisiti informali abbiamo ricavato un insieme di requisiti strutturati, organizzati in requisiti funzionali (RF), non funzionali (RN) e di dominio (RD). Il diagramma dei requisiti li raccoglie tutti.



Requisiti funzionali

ID	Nome	Descrizione
RF01	AutenticazioneUtenti	Il sistema prevede un meccanismo di autenticazione, che associa un profilo ad ogni utente.
RF02	GestioneProdottiAmm	Gli amministratori possono gestire i prodotti del catalogo creandoli, modificandoli e

		rimuovendoli. Un prodotto può anche essere reso non disponibile, quindi non acquistabile.
RF03	VisualizzaProdottiUtente	Gli utenti possono visualizzare il catalogo dei prodotti e le informazioni annesse a ognuno.
RF04	GestioneCarrelloUtente	Gli utenti possono aggiungere i prodotti selezionati al carrello, senza poter acquistare quantità superiori a quelle disponibili.
RF05	GestioneOrdiniUtente	Gli utenti possono confermare l'ordine di un carrello e consultare informazioni sui loro ordini.
RF06	RicercaProdotti	Il sistema prevede una funzione di ricerca dei prodotti secondo i loro parametri caratteristici.
RF07	GestioneOrdiniAmm	Gli amministratori aggiornano lo stato degli ordini.
RF08	GestioneStatisticheAmm	Gli amministratori possono monitorare statistiche della piattaforma.
RF09	StoricoOrdiniUtente	Il sistema memorizza gli ordini di un utente e i prodotti acquistati nel tempo.

Requisiti non funzionali

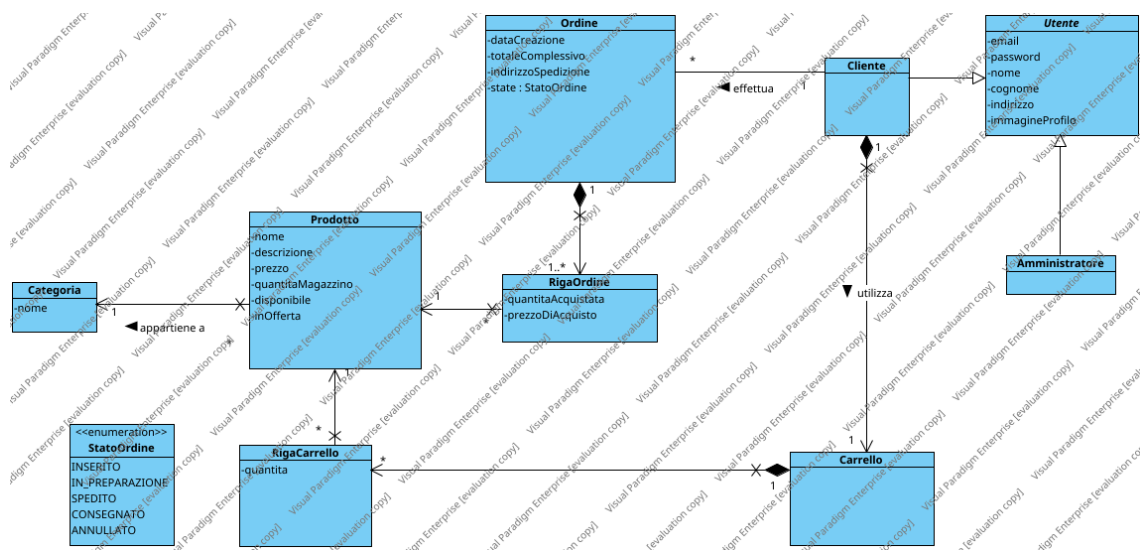
ID	Nome	Descrizione
RN01	SupportoPiattaforme	Il sistema deve essere multipiattaforma.
RN02	GestioneNotifiche	Il sistema deve prevedere un sistema di notifiche per gli aggiornamenti dello stato dell'ordine.
RN03	GestioneSicurezza	Il sistema deve prevedere standard di sicurezza che riguardano la protezione dei dati personali e l'accesso.

Requisiti di dominio

ID	Nome	Descrizione
RD01	Prodotto	Il sistema prevede attributi caratteristici per i prodotti e la possibilità di renderli non acquistabili per indisponibilità.
RD02	Ordine	Il sistema prevede attributi caratteristici dell'ordine e gli stati che esso può assumere (inserito, in preparazione, spedito, consegnato).

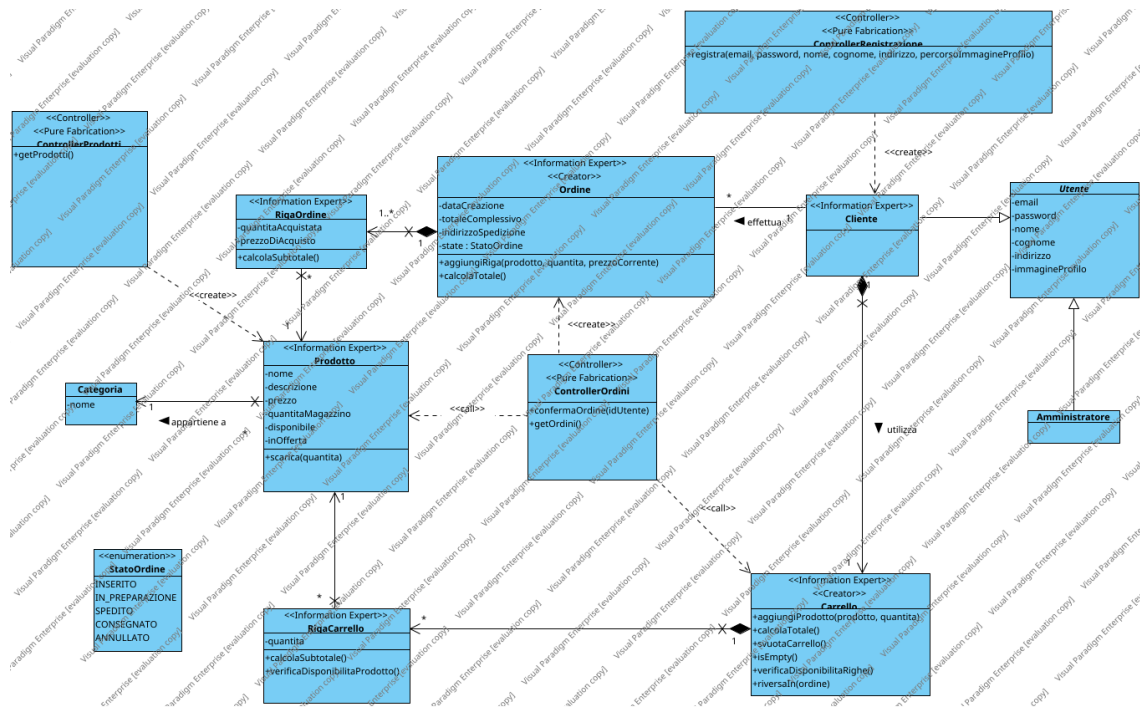
2.2 Modello di dominio

Dall'analisi dei requisiti abbiamo estratto i concetti principali del dominio e le loro relazioni. Il modello di dominio non contiene ancora scelte implementative: descrive solo le entità in gioco e i legami tra loro. I concetti centrali sono l'Utente, con le sue due specializzazioni Cliente e Amministratore, il Prodotto con la sua Categoria, il Carrello con le sue righe, l'Ordine con le sue righe e lo stato.



Un Cliente possiede un solo Carrello, e un Carrello appartiene a un solo Cliente. Il Carrello è composto da più righe, ognuna delle quali fa riferimento a un Prodotto e a una quantità. Un Cliente può avere più Ordini nel tempo, e ogni Ordine è composto da almeno una riga, che congela il prodotto acquistato, la quantità e il prezzo al momento dell'acquisto. Ogni Prodotto appartiene a una Categoria. L'Ordine ha uno stato che cambia nel tempo, descritto più avanti dalla macchina a stati.

Abbiamo poi arricchito il modello di dominio con le responsabilità, assegnando a ciascuna classe i compiti che le competono. Questo passaggio serve da ponte verso il design: qui iniziano a comparire le scelte su chi fa cosa, seguendo i principi GRASP.



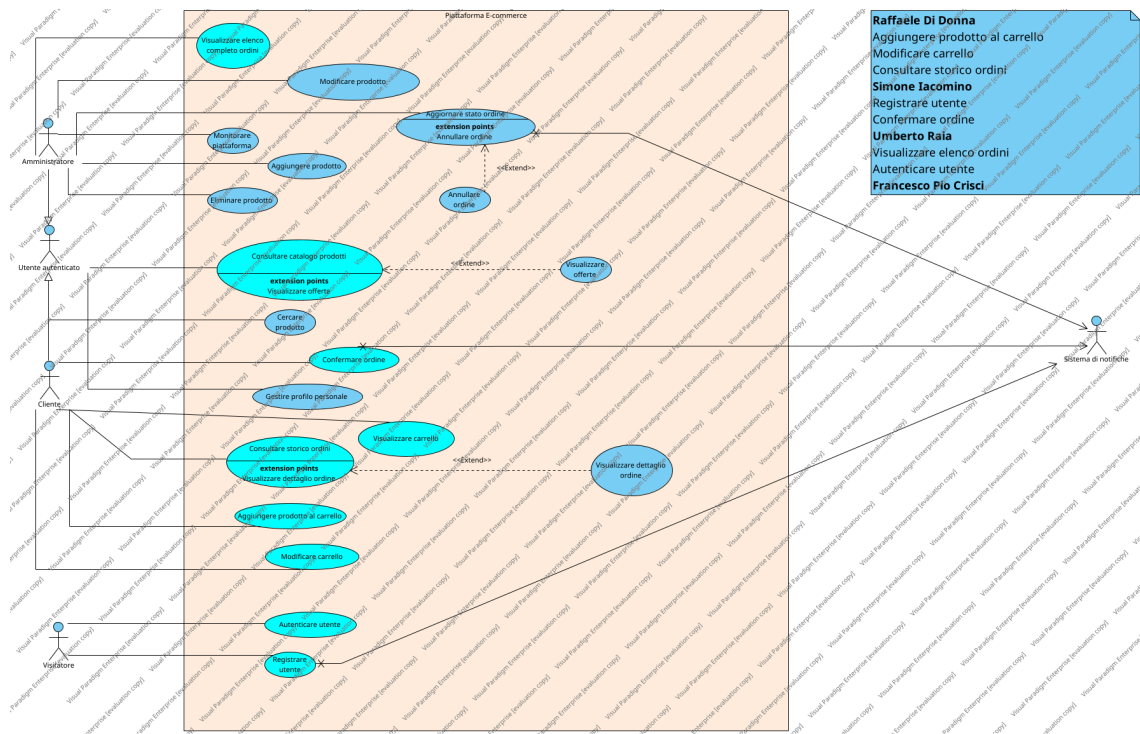
2.3 Attori

Il sistema riconosce i seguenti attori:

- **Visitatore**: utente non ancora autenticato, può solo registrarsi o autenticarsi. - **Utente autenticato**: generalizzazione di Cliente e Amministratore, raccoglie ciò che è comune ai due ruoli una volta fatto l'accesso. - **Cliente**: utente autenticato che acquista. - **Amministratore**: utente autenticato che gestisce prodotti e ordini della piattaforma. - **Sistema di notifiche**: attore di sistema che riceve le richieste di invio notifica in occasione di eventi come la conferma di un ordine o la registrazione di un utente.

2.4 Casi d'uso

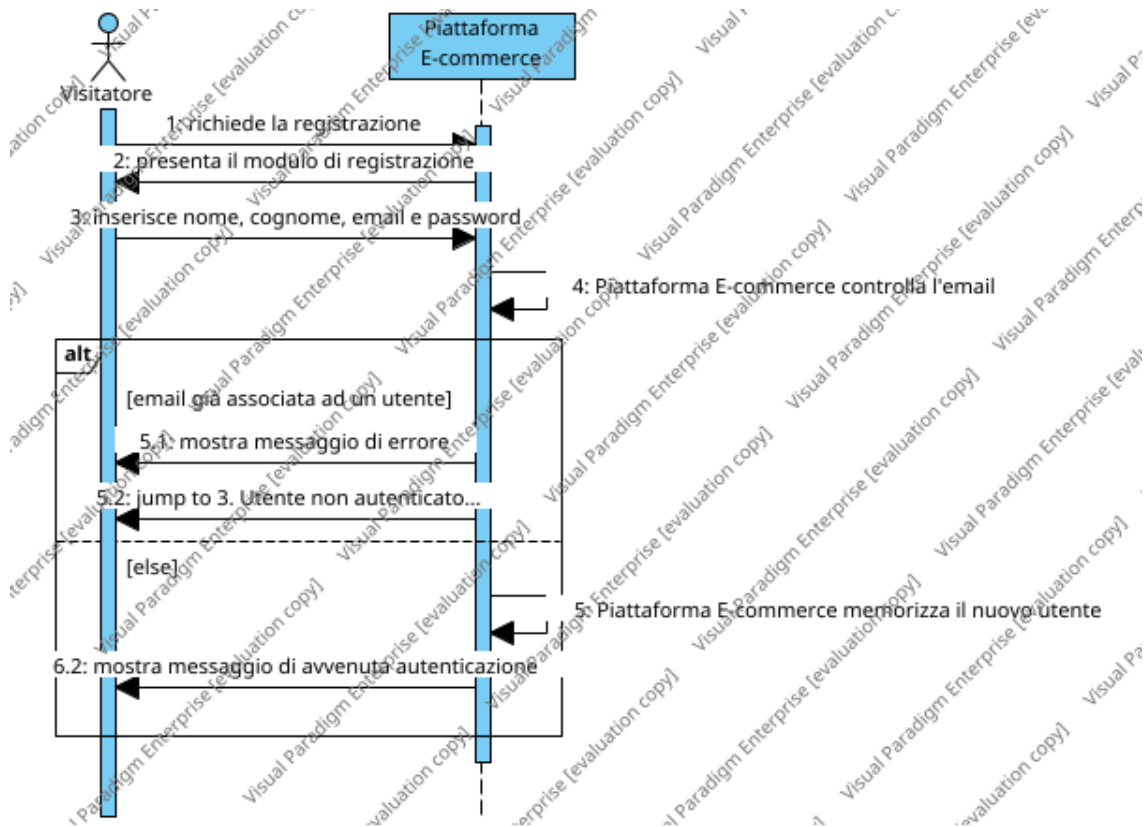
Il diagramma dei casi d'uso raccoglie tutte le funzionalità individuate in analisi, comprese quelle che in questa versione del sistema sono state modellate ma non ancora realizzate. Le funzionalità effettivamente implementate sono poi riprese nella sezione 3.



Di seguito riportiamo i casi d'uso con il loro flusso degli eventi, così come descritto nel modello. Per i casi d'uso principali, ognuno è corredato nel modello VP di un diagramma di sequenza (scenario) e di una use case description dettagliata, che possono essere inseriti dopo la relativa descrizione testuale.

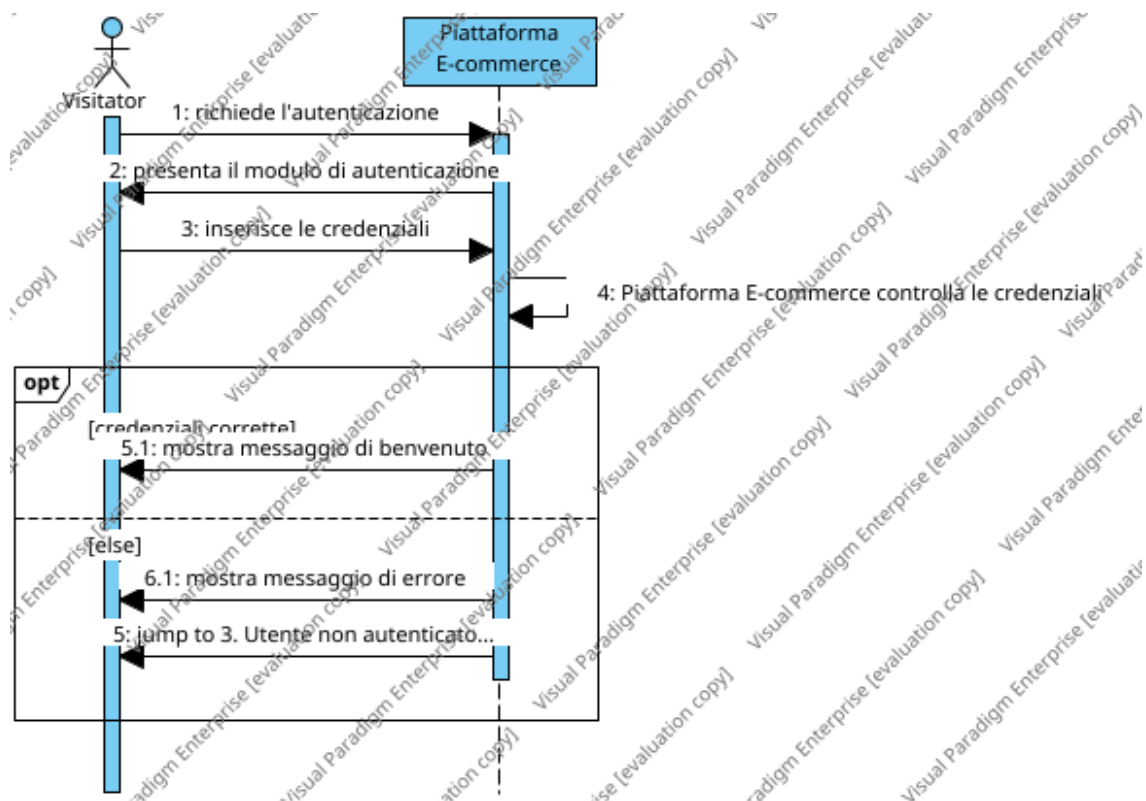
Registrare utente

1. Il Visitatore richiede la registrazione e la piattaforma presenta il modulo.
2. Il Visitatore inserisce nome, cognome, email, password, indirizzo e immagine del profilo.
3. La piattaforma controlla l'email.
4. Se l'email è già associata a un utente, mostra un messaggio di errore.
5. Altrimenti memorizza il nuovo utente e mostra un messaggio di conferma.



Autenticare utente

1. Il Visitatore richiede l'autenticazione e la piattaforma presenta il modulo. 2. Il Visitatore inserisce le credenziali. 3. La piattaforma controlla le credenziali. 4. Se sono corrette, mostra la pagina principale. 5. Altrimenti mostra un messaggio di errore.

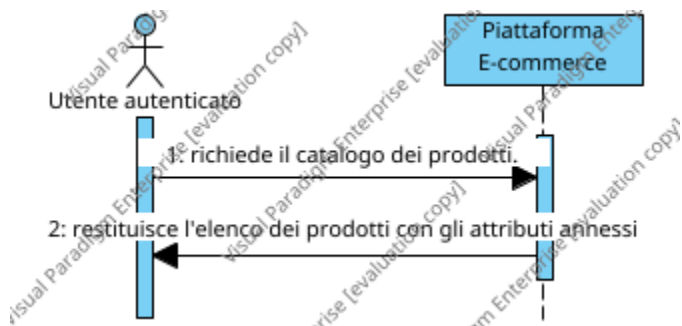


Gestire profilo personale

1. L'Utente autenticato richiede l'accesso al proprio profilo. 2. La piattaforma mostra le informazioni del profilo personale. 3. L'Utente fornisce le modifiche desiderate. 4. La piattaforma valida i dati forniti e presenta un messaggio di conferma.

Consultare catalogo prodotti

1. L'Utente autenticato richiede il catalogo dei prodotti. 2. La piattaforma restituisce l'elenco dei prodotti disponibili con gli attributi annessi.



Cercare prodotto (extend di Consultare catalogo prodotti)

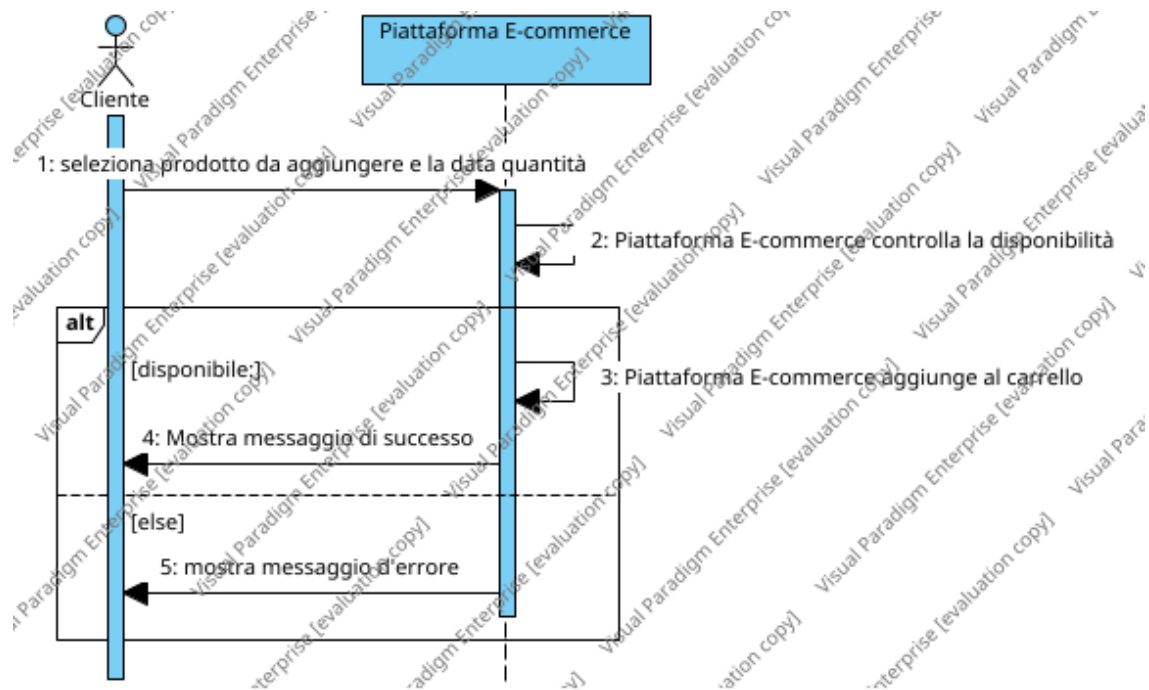
1. L'Utente autenticato richiede un prodotto in particolare. 2. La piattaforma ricerca i prodotti corrispondenti. 3. Se ce n'è almeno uno, li mostra. 4. Altrimenti mostra un messaggio di errore.

Visualizzare offerte (extend di Consultare catalogo prodotti)

1. L'Utente autenticato accede alla sezione "Offerte". 2. La piattaforma ricerca i prodotti contrassegnati come in offerta. 3. Se ce n'è almeno uno, li mostra. 4. Altrimenti mostra un messaggio di errore.

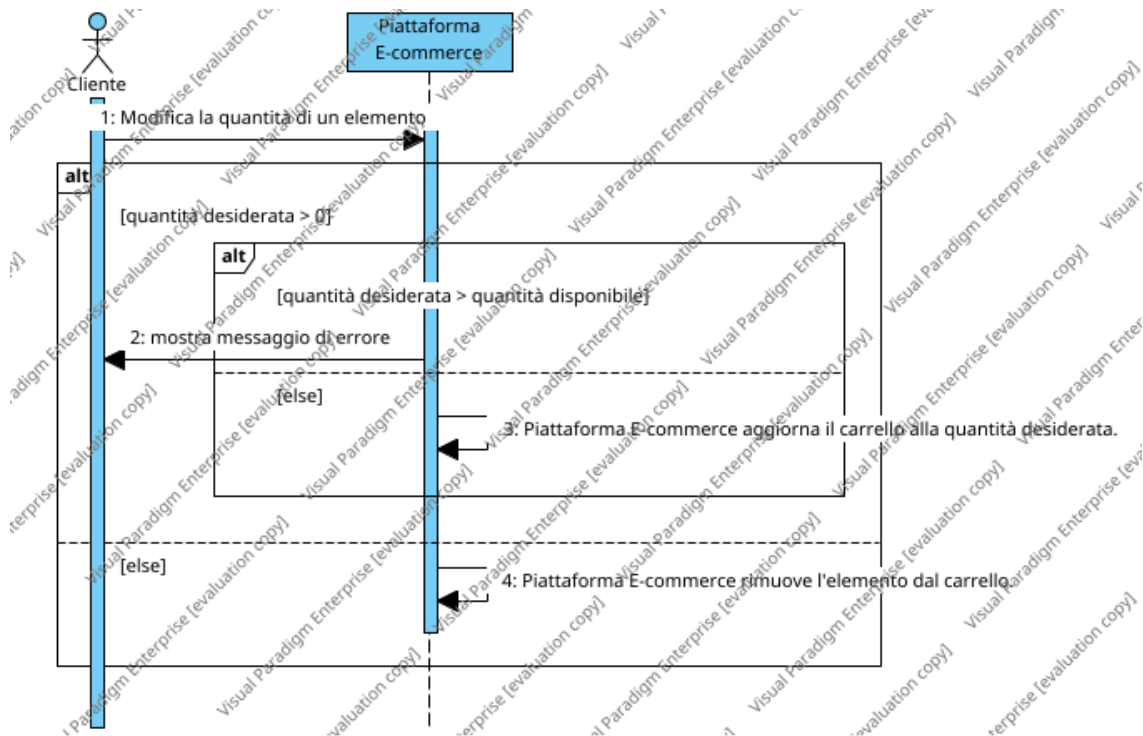
Aggiungere prodotto al carrello

1. Il Cliente seleziona il prodotto da aggiungere e la relativa quantità. 2. La piattaforma controlla la disponibilità. 3. Se il prodotto è disponibile nella quantità richiesta, lo aggiunge al carrello e mostra un messaggio di successo. 4. Altrimenti mostra un messaggio di errore.



Modificare carrello

1. Il Cliente modifica la quantità di un elemento. 2. Se la quantità desiderata è maggiore di zero: - se è maggiore della quantità disponibile, mostra un messaggio di errore; - altrimenti aggiorna il carrello alla quantità desiderata. 3. Se la quantità desiderata è zero, rimuove l'elemento dal carrello. 4. Altrimenti mostra un messaggio di errore.



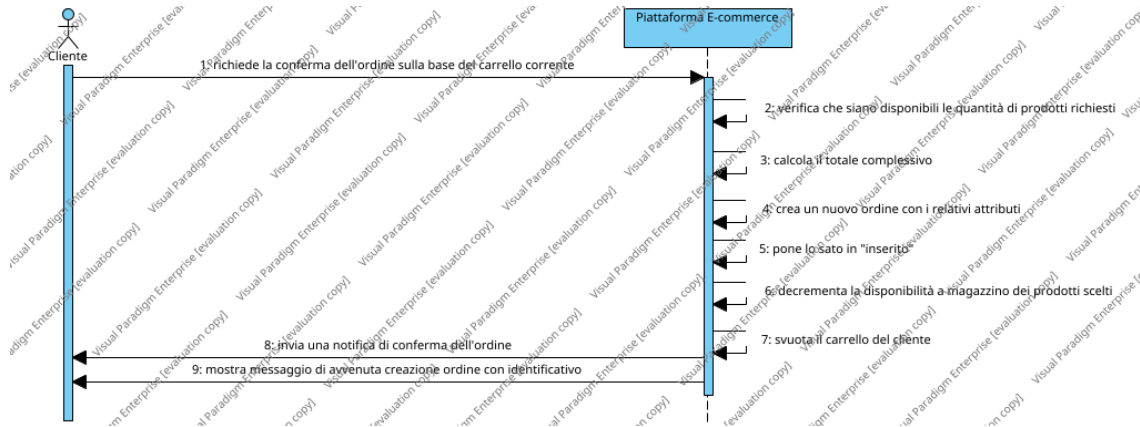
Visualizzare carrello

Il Cliente richiede la visualizzazione del proprio carrello e la piattaforma ne mostra il contenuto con i subtotali di riga e il totale complessivo.

Confermare ordine

1. Il Cliente richiede la conferma dell'ordine sulla base del carrello corrente. 2. La piattaforma verifica che siano disponibili le quantità richieste. 3. Calcola il totale complessivo. 4. Crea un nuovo ordine con i prodotti del carrello, le rispettive quantità, il totale e la data di creazione. 5. Pone lo stato dell'ordine in "inserito". 6. Decrementa la disponibilità a magazzino di ciascun prodotto per la quantità ordinata. 7. Svuota il carrello del Cliente. 8. Invia al Cliente una notifica di conferma dell'ordine. 9. Mostra un messaggio di avvenuta creazione dell'ordine.

Scenario di indisponibilità: se la piattaforma rileva che la disponibilità di qualche prodotto è insufficiente, mostra un messaggio di errore senza creare l'ordine.



Consultare storico ordini

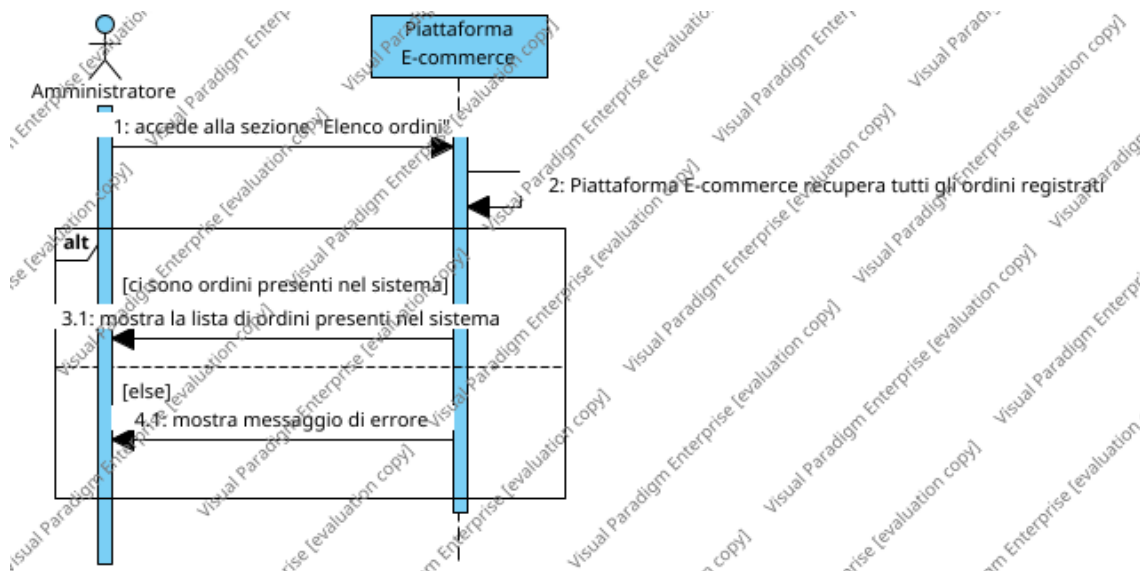
1. Il Cliente richiede la visione dello storico dei propri ordini. 2. La piattaforma recupera gli ordini effettuati dall'utente. 3. Se ne esiste almeno uno, presenta la lista con data, totale e stato corrente di ognuno. 4. Altrimenti presenta un messaggio di esito nullo.

Visualizzare dettaglio ordine (extend di Consultare storico ordini)

1. Il Cliente richiede i dettagli di un ordine specifico. 2. La piattaforma recupera i dati di dettaglio dell'ordine selezionato. 3. Mostra il dettaglio completo dell'ordine.

Visualizzare elenco completo ordini

1. L'Amministratore accede alla sezione "Elenco ordini". 2. La piattaforma recupera tutti gli ordini registrati. 3. Se ci sono ordini, mostra la lista di tutti gli ordini presenti nel sistema. 4. Altrimenti mostra un messaggio di errore.



Aggiornare stato ordine

1. L'Amministratore seleziona un ordine e la piattaforma ne presenta i dettagli e lo stato attuale.
2. L'Amministratore seleziona il nuovo stato desiderato e conferma.
3. La piattaforma registra il nuovo stato e presenta un messaggio di conferma.

Annullare ordine

1. La piattaforma ripristina le quantità dei prodotti associati all'ordine.
2. Registra il nuovo stato come annullato.
3. Presenta il messaggio di avvenuto annullamento.

Monitorare piattaforma

1. L'Amministratore accede alla sezione "Monitoraggio Piattaforma".
2. La piattaforma recupera i dati e calcola le statistiche.
3. Mostra una dashboard con i grafici dei dati.

Aggiungere prodotto

1. L'Amministratore inserisce i dati del nuovo prodotto.
2. Se il prodotto esiste già, o la quantità è negativa, o la categoria non è valida, mostra un messaggio di errore.
3. Altrimenti registra il prodotto e ne mostra la creazione.

Modificare prodotto

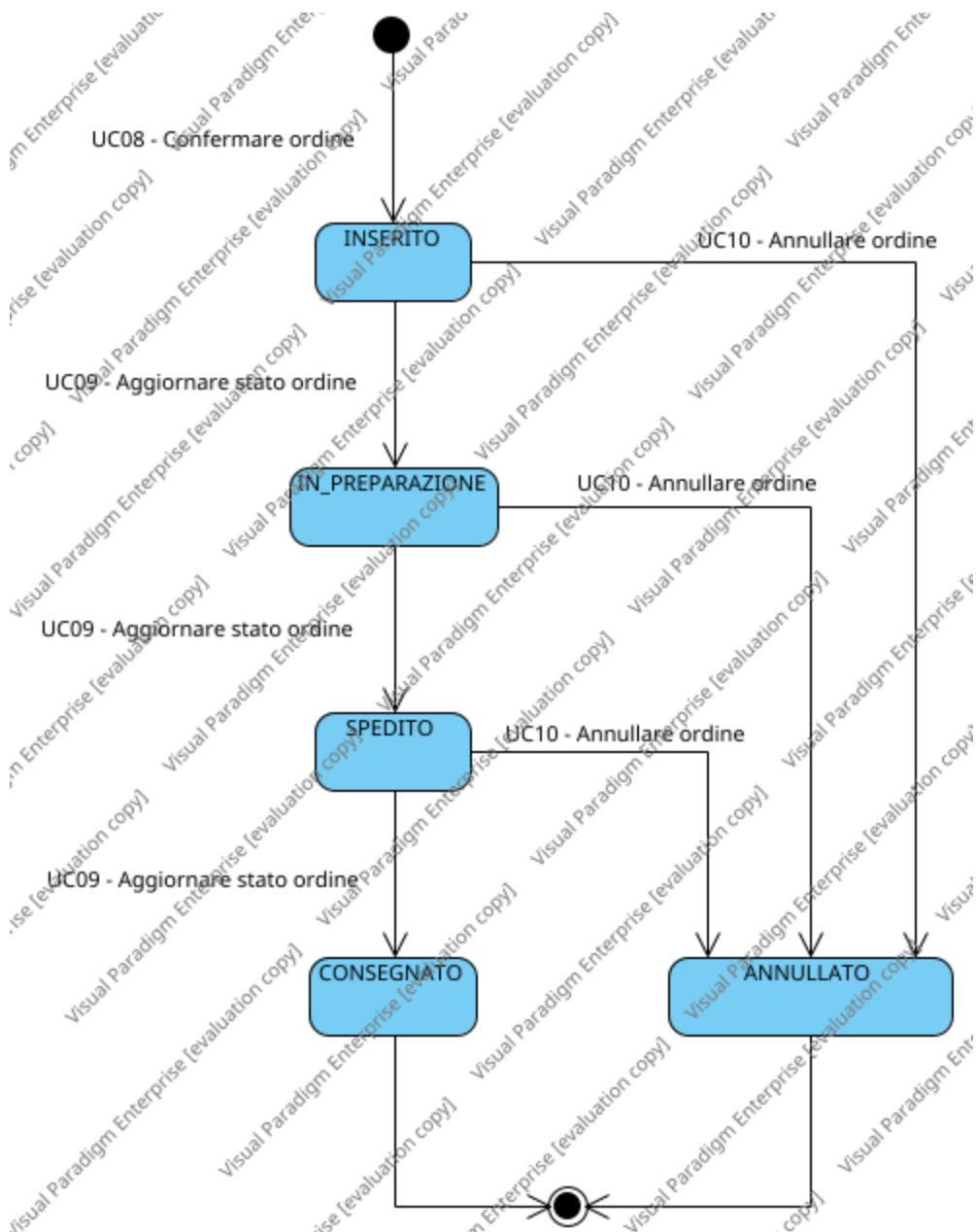
1. L'Amministratore seleziona il prodotto e la piattaforma ne restituisce la scheda.
2. L'Amministratore modifica i campi interessati.
3. Se la quantità è negativa o la categoria non è valida, mostra un messaggio di errore.
4. Altrimenti memorizza i nuovi dati e invia una conferma con i dettagli aggiornati.

Eliminare prodotto

1. L'Amministratore seleziona il prodotto da eliminare.
2. La piattaforma lo elimina dal catalogo.
3. Mostra un messaggio di successo.

2.5 Ciclo di vita dell'ordine

Il caso d'uso "Confermare ordine" e i casi d'uso di gestione ordine da parte dell'amministratore agiscono sullo stato dell'ordine. La macchina a stati ne descrive il ciclo di vita: alla conferma l'ordine entra nello stato "inserito", da cui passa a "in preparazione", "spedito" e infine "consegnato", che è lo stato finale. Dagli stati intermedi è possibile passare ad "annullato", anch'esso finale, ripristinando le quantità a magazzino.



2.6 Tracciabilità requisiti / casi d'uso

La tabella lega ogni caso d'uso ai requisiti che contribuisce a soddisfare, così da verificare che ogni requisito funzionale sia coperto da almeno un caso d'uso.

Caso d'uso	Requisiti coperti
Registrare utente	RF01, RN03
Autenticare utente	RF01, RN03
Gestire profilo personale	RF01
Consultare catalogo prodotti	RF03
Cercare prodotto	RF06
Visualizzare offerte	RF03

Aggiungere prodotto al carrello	RF04, RD01
Modificare carrello	RF04
Visualizzare carrello	RF04
Confermare ordine	RF05, RN02, RD02
Consultare storico ordini	RF05, RF09
Visualizzare dettaglio ordine	RF09
Visualizzare elenco completo ordini	RF07
Aggiornare stato ordine	RF07, RN02, RD02
Annullare ordine	RF07, RD02
Monitorare piattaforma	RF08
Aggiungere prodotto	RF02, RD01
Modificare prodotto	RF02, RD01
Eliminare prodotto	RF02

3. Design del Sistema Software Implementato

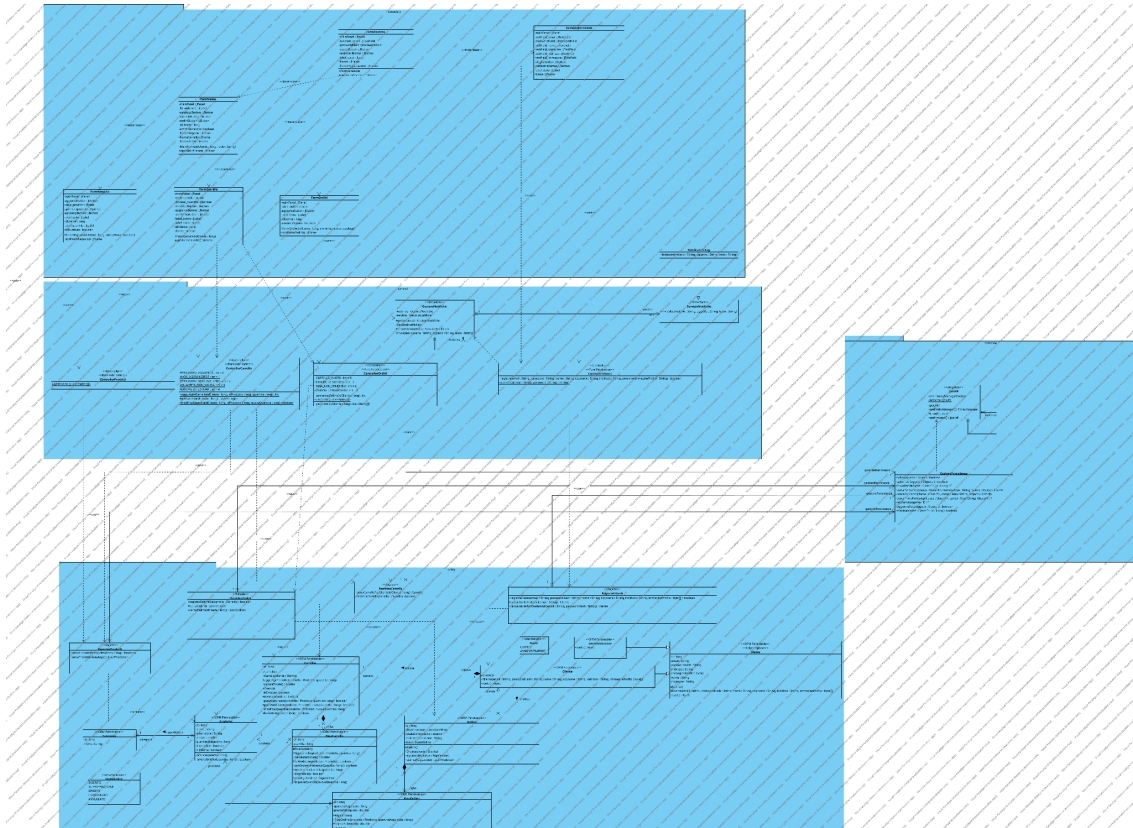
Questa sezione descrive l'architettura del software effettivamente realizzato e il modo in cui vengono eseguiti i casi d'uso implementati. Rispetto all'analisi, che copre l'intero modello, qui ci concentriamo sulle funzionalità che sono state portate a codice: registrazione, autenticazione, consultazione del catalogo, gestione del carrello (aggiunta, visualizzazione, modifica), conferma dell'ordine, consultazione dello storico ordini e visualizzazione dell'elenco completo degli ordini.

3.1 Architettura generale

Il sistema segue un approccio a livelli di tipo BCED (Boundary, Control, Entity, Database). Ogni livello corrisponde a un package Java e ha una responsabilità precisa.

- **boundary**: contiene l'interfaccia grafica realizzata con Swing (i vari Form e il MainFrame). Si occupa solo di presentare i dati e raccogliere gli input dell'utente, delegando ogni logica ai controller. - **control**: contiene i controller, che coordinano l'esecuzione dei casi d'uso. Ricevono le richieste dalla boundary, orchestrano le entità e restituiscono alla boundary dati semplici (array di stringhe o codici di esito). - **entity**: contiene le classi di dominio (Utente, Cliente, Amministratore, Prodotto, Carrello, Ordine e le relative righe) più i registri, che fanno da punto d'accesso unico a ciascun aggregato. - **database**: contiene l'infrastruttura di persistenza basata su JPA/Hibernate (GestorePersistenza e JpaUtil).

Il modello di sistema riportato nel class diagram mostra tutte le classi con i loro package e le dipendenze tra strati.



La regola che abbiamo seguito è che le dipendenze vanno in una sola direzione: la boundary conosce il control, il control conosce le entity, le entity (tramite i registri) conoscono il database. Questo tiene la logica applicativa separata dalla presentazione, cosa che ha reso possibile testare i controller senza avviare l'interfaccia grafica (vedi sezione 5).

3.2 Design pattern adottati

Nel design abbiamo applicato alcuni pattern, sia GRASP sia GoF, che spieghiamo di seguito con i punti del codice in cui compaiono.

Controller (GRASP). Ogni caso d'uso è affidato a un controller dedicato (ControllerUtenti, ControllerProdotti, ControllerCarrello, ControllerOrdini). Il controller non contiene logica di dominio, ma la coordina. Per esempio la conferma di un ordine si riduce a una sequenza di verifiche e a una delega ai registri:

```
public static int confermaOrdine(long idCliente) {
    RegistroCarrello reg_c = new RegistroCarrello();
    RegistroOrdini reg_o = new RegistroOrdini();
    Carrello carrello = reg_c.cercaCarrelloPerCliente(idCliente);

    if (carrello == null) {return ERRORE_DI_SISTEMA;}
    if (carrello.isEmpty()) {return CARRELLO_VUOTO;}
    if (!carrello.haTutteRigheVendibili()) {return RIGA_NON_VENDIBILE;}
}
```

```

        boolean esito = reg_o.registraOrdineDa(carrello);
        if(!esito){return ERRORE_DI_SISTEMA;}

        String email = carrello.getClient().getEmail();
        GestoreNotifiche.getInstance().invia(email, "Ordine confermato",
            "Il tuo ordine è stato registrato correttamente.");

        return ORDINE_CONFERMATO;
    }

```

Information Expert e Creator (GRASP). La logica di dominio sta nelle entità che possiedono i dati necessari. È il Prodotto a sapere se può essere venduto in una certa quantità, ed è il Carrello a sapere se un prodotto può essere aggiunto tenendo conto di quanto già presente:

```

public boolean isVendibilePer(long quantita) {
    return disponibile && quantita > 0 && quantita <= quantitaMagazzino;
}

```

```

public boolean puoAggiungere(Prodotto prodotto, long quantita) {
    long totale = quantitaInCarrello(prodotto) + quantita;
    return prodotto.isVendibilePer(totale);
}

```

Il Carrello è anche il Creator delle proprie righe (crea la RigaCarrello quando serve), e l'Ordine è il Creator delle proprie RigaOrdine.

Facade (GoF). I registri (RegistroUtenti, RegistroProdotti, RegistroCarrello, RegistroOrdini) fanno da facciata verso la persistenza: offrono ai controller metodi semplici e nascondono il dettaglio delle query. Il controller non sa nulla di JPA, chiede solo "cercami il carrello di questo cliente".

Polymorphism (GRASP). La classe astratta Utente definisce il metodo ruolo(), implementato in modo diverso da Cliente e Amministratore. Chi legge il ruolo di un utente non deve fare controlli sul tipo: chiede ruolo() e ottiene la risposta corretta.

Singleton (GoF). Sia JpaUtil sia GestoreNotifiche sono singleton. JpaUtil mantiene l'unica EntityManagerFactory dell'applicazione, che è un oggetto costoso da creare e va condiviso:

```

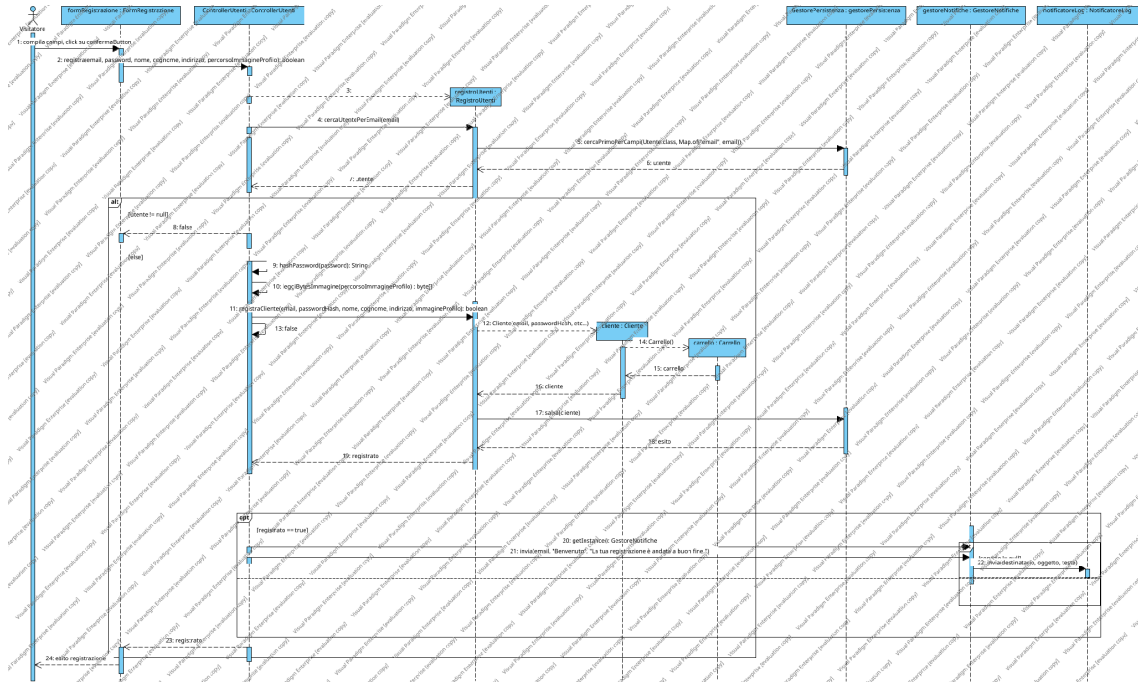
public static JpaUtil getInstance() {
    if (instance == null) {
        instance = new JpaUtil();
    }
    return instance;
}

```

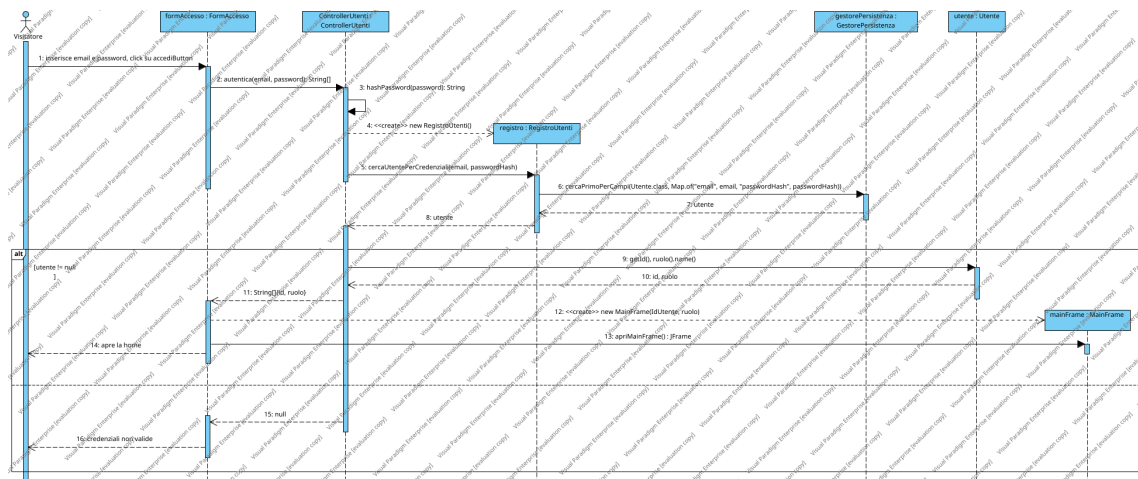
3.3 Dinamica di esecuzione dei casi d'uso

Per documentare come i vari oggetti collaborano a runtime abbiamo prodotto un sequence diagram per ciascun caso d'uso implementato. In ogni diagramma si vede lo stesso schema di collaborazione: il Form della boundary chiama il metodo statico del controller, il controller interroga i registri, i registri passano dal GestorePersistenza, e le entità applicano la logica di dominio.

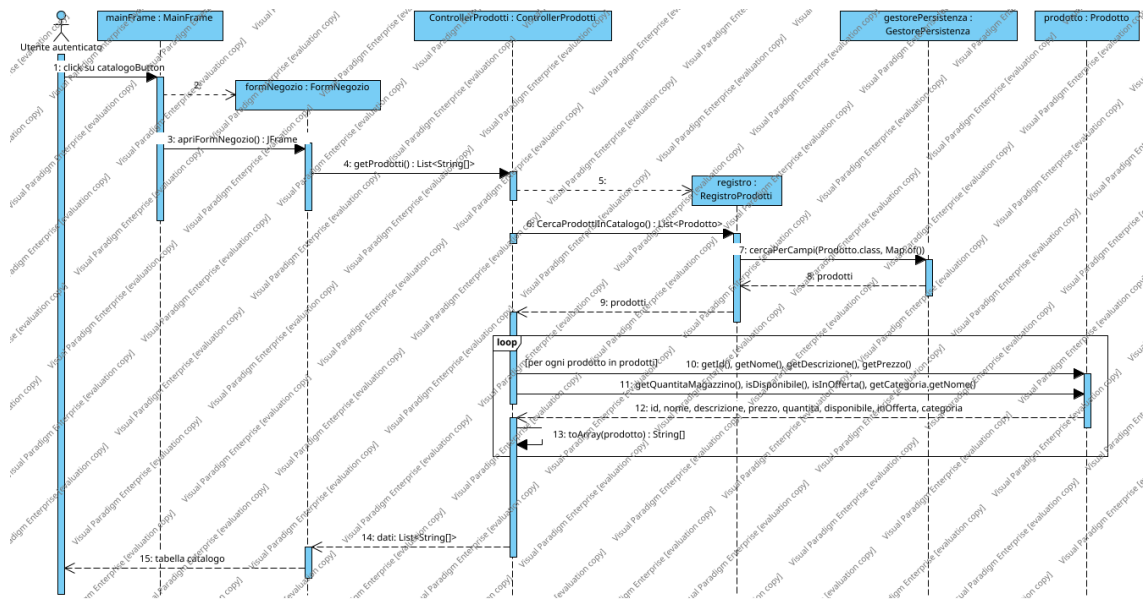
Caso d'uso "Registrare utente":



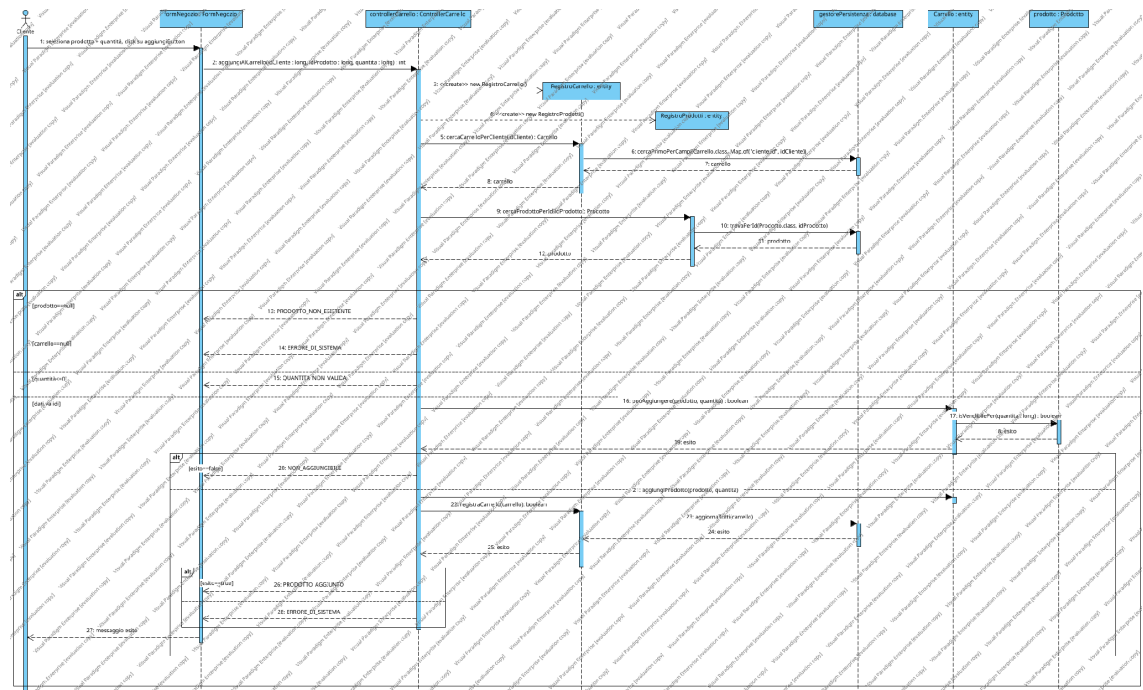
Caso d'uso "Autenticare utente":



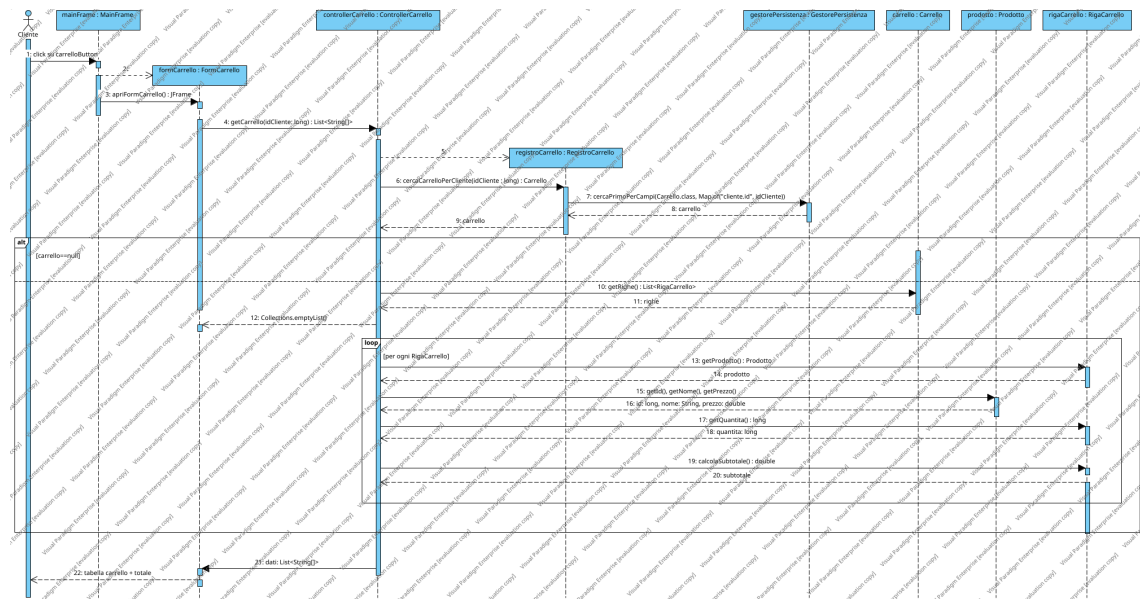
Caso d'uso "Consultare catalogo prodotti":



Caso d'uso "Aggiungere prodotto al carrello":



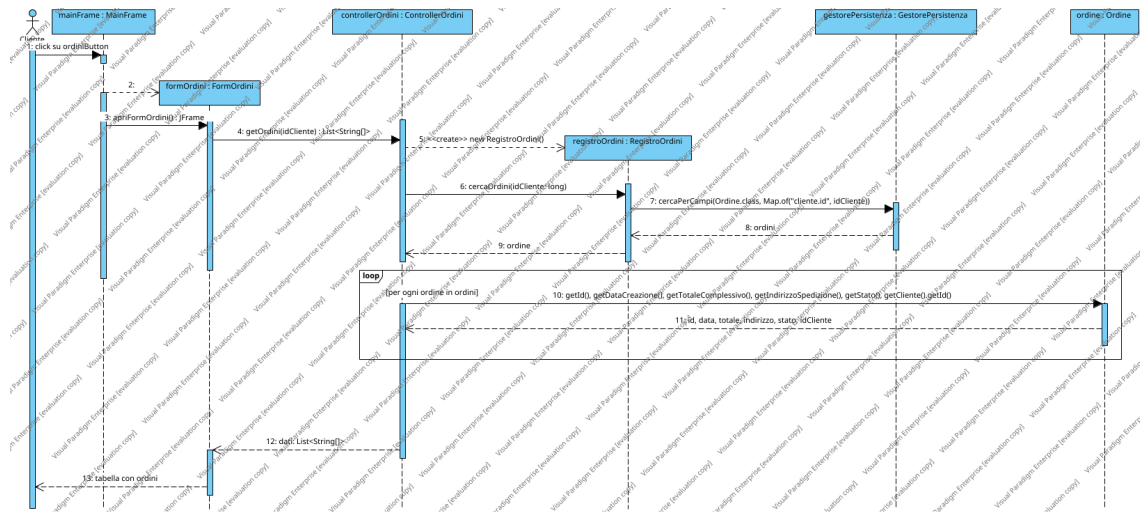
Caso d'uso "Visualizzare carrello":



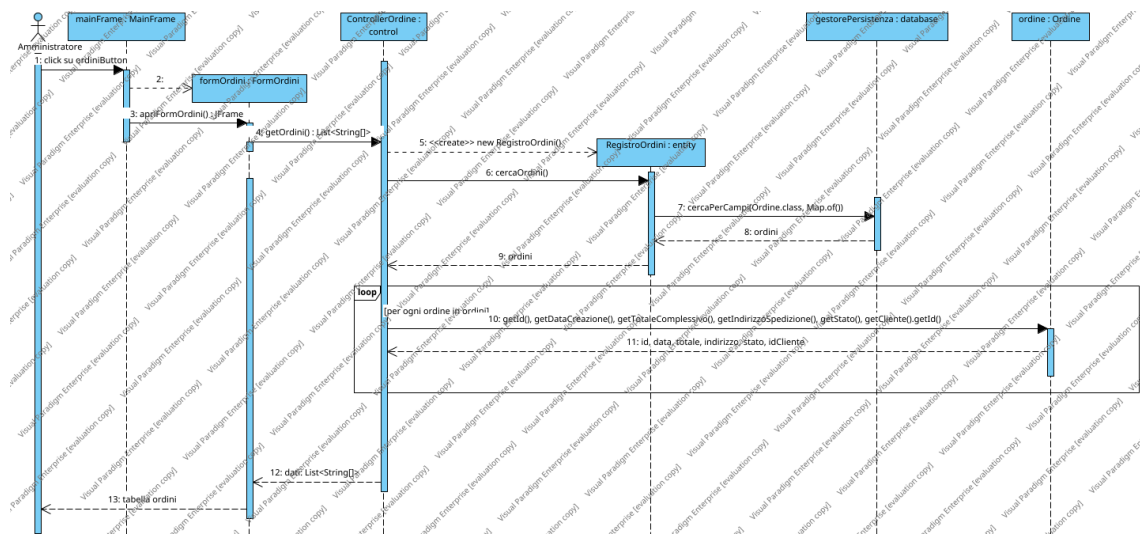
Caso d'uso "Confermare ordine":



Caso d'uso "Consultare storico ordini":



Caso d'uso "Visualizzare elenco completo ordini":



Prendiamo come esempio la conferma dell'ordine, che è il caso d'uso più articolato. Il FormCarrello chiama `ControllerOrdini.confermaOrdine`. Il controller recupera il carrello tramite il `RegistroCarrello` e fa le tre verifiche di guardia (carrello esistente, non vuoto, con tutte le righe ancora vendibili). Se tutto è a posto delega al `RegistroOrdini`, che crea l'ordine dal carrello, ne riversa le righe, scarica il magazzino e svuota il carrello in una sola transazione di persistenza. Solo a esito positivo il controller chiede al `GestoreNotifiche` di inviare la notifica di conferma.

3.4 Tracciabilità design / casi d'uso

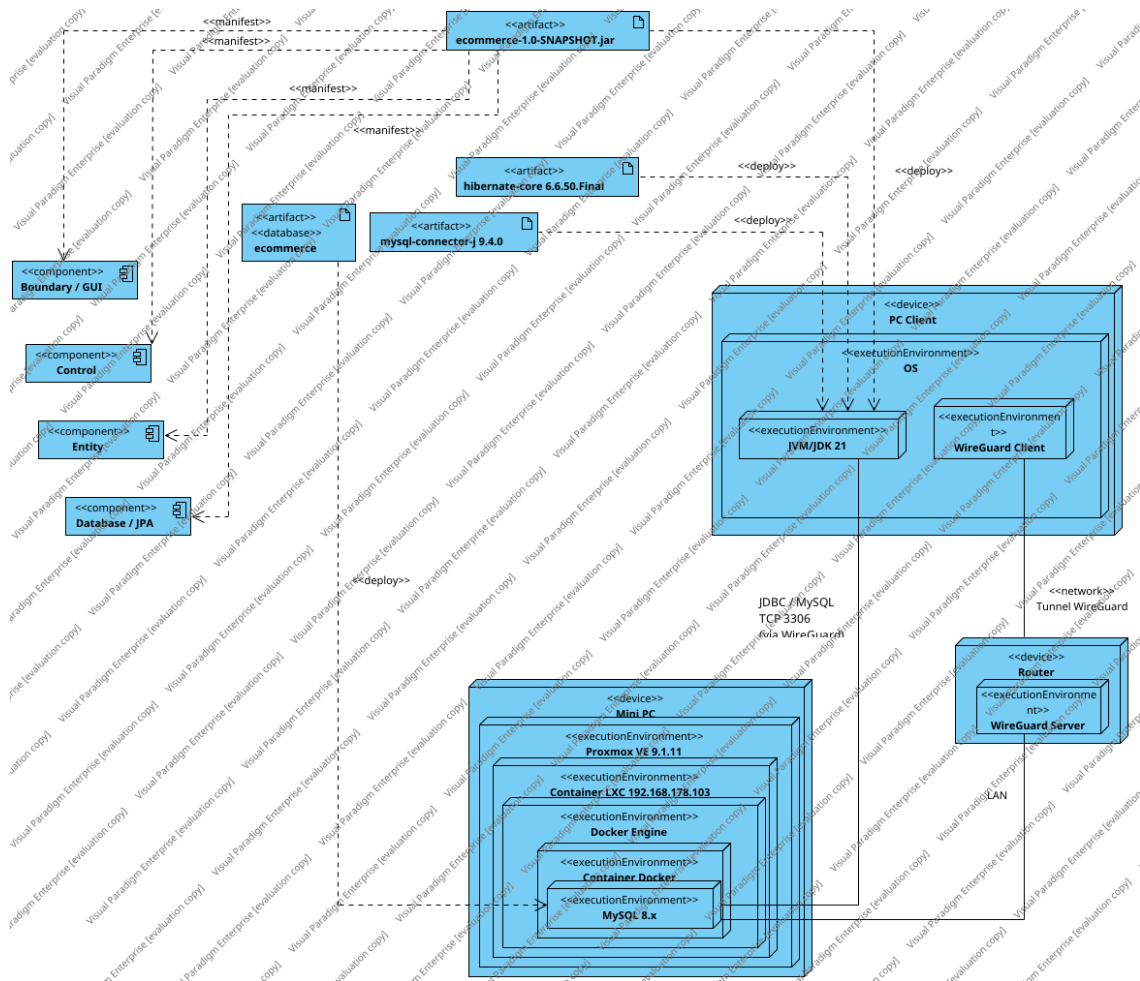
La tabella lega ogni caso d'uso implementato al metodo di controllo che lo realizza, così da verificare la copertura delle funzionalità realizzate.

Caso d'uso	Metodo di controllo	Requisiti
------------	---------------------	-----------

Registrare utente	ControllerUtenti.registra	RF01, RN02, RN03
Autenticare utente	ControllerUtenti.autentica	RF01, RN03
Consultare catalogo prodotti	ControllerProdotti.getProdotti	RF03
Aggiungere prodotto al carrello	ControllerCarrello.aggiungiAlCarrello	RF04, RD01
Visualizzare carrello	ControllerCarrello.getCarrello	RF04
Modificare carrello	ControllerCarrello.modificaQuantita	RF04
Confermare ordine	ControllerOrdini.confermaOrdine	RF05, RN02, RD02
Consultare storico ordini	ControllerOrdini.getOrdini(id Cliente)	RF05, RF09
Visualizzare elenco completo ordini	ControllerOrdini.getOrdini()	RF07

4. Diagramma di Deploy

Il sistema è un'applicazione desktop che accede a un database remoto. Il diagramma di deploy mostra l'ambiente di esecuzione reale usato dal gruppo, con la parte client e la parte server separate e collegate da una rete privata.



Lato client, l'applicazione gira su un normale PC dell'utente. Sul PC è installato un sistema operativo (Windows, Linux o macOS, coerentemente con il requisito RN01 di multiplatforma), sopra il quale gira la JVM (JDK 21). La JVM esegue l'artefatto dell'applicazione, `ecommerce-1.0-SNAPSHOT.jar`, che al suo interno contiene i quattro componenti logici corrispondenti agli strati dell'architettura (boundary, control, entity, database) più le librerie da cui dipende, in particolare Hibernate e il connettore MySQL.

Lato server, il database non gira su una macchina dedicata ma dentro una catena di virtualizzazione. Il nodo fisico è un mini PC su cui è installato Proxmox VE come ambiente di virtualizzazione. Dentro Proxmox c'è un container LXC, raggiungibile all'indirizzo `192.168.178.103`, che a sua volta ospita un Docker Engine. Il container Docker esegue MySQL 8.x, dentro cui vive il database `ecommerce`.

Il collegamento tra client e server passa per una rete privata WireGuard: il client si connette al server tramite un router e un tunnel VPN, quindi il database non è esposto direttamente su Internet ma è raggiungibile solo da chi è dentro la rete privata del gruppo. Questa scelta spiega perché, per eseguire l'applicazione con il database reale, serve avere la VPN attiva.

5. Test Plan

Questa sezione descrive il piano di test previsto per verificare il corretto funzionamento del software. Indichiamo l'approccio seguito, i criteri di progettazione dei casi di test, le funzionalità verificate, i risultati di copertura e la tracciabilità tra test, casi d'uso e requisiti. Gli identificatori dei casi di test (TCBB per i black-box, TCWB per i white-box) si ritrovano nei `@DisplayName` dei metodi JUnit, così ogni caso di test del documento è rintracciabile nel codice e viceversa.

5.1 Approccio: prima black-box, poi white-box

La strategia seguita è quella prescritta dagli appunti del corso: prima si progetta una suite con tecniche black-box a partire dalle specifiche, poi si misura la copertura strutturale ottenuta e infine si aggiungono casi di test white-box per coprire gli elementi rimasti scoperti. I due approcci non sono alternativi. Il black-box dice cosa testare partendo dalle specifiche, quindi è capace di far emergere comportamenti previsti dal modello ma dimenticati nel codice. Il white-box dice se abbiamo testato abbastanza partendo dal codice, quindi mette in luce il codice che nessuna specifica descrive.

Derivare i casi di test dalle specifiche e non dai sorgenti è una scelta precisa: se partissimo dal codice, i test erediterebbero gli stessi eventuali errori del codice, e finiremmo per verificare che il programma fa quello che fa invece di quello che deve fare.

5.2 Come sono stati progettati i casi di test

I casi di test black-box nascono da una catena fissa di quattro passaggi, uguale per ogni caso d'uso:

flusso degli eventi → classi di equivalenza → casi di test → metodo JUnit

Le condizioni che il flusso pone sugli input diventano le classi di equivalenza. Ogni classe genera almeno un caso di test, e ogni caso di test diventa un metodo JUnit con il proprio identificatore nel `@DisplayName`. Per le classi di equivalenza valide abbiamo usato il testing combinato, mentre per le classi non valide abbiamo progettato casi di test separati, in modo da capire subito quale input ha fatto fallire il test. Dove un input è un numero con un intervallo di validità abbiamo aggiunto l'analisi dei valori limite.

Ogni caso di test è definito da uno stato iniziale del sistema, dalle precondizioni, dall'azione eseguita e dall'output atteso. Questa è esattamente la forma dei metodi JUnit: prima si costruisce lo stato, poi si agisce, poi si verifica. Un esempio:

```
@Test
@DisplayName("TCBB16 - Aggiunta oltre il limite superiore")
void aggiuntaOltreIlLimiteSuperiore() {
    Cliente cliente = creaCliente("mario@test.it", "password");
    Prodotto mouse = creaProdotto("Mouse", 25.0, 10, true);

    assertEquals(ControllerCarrello.NON_AGGIUNGIBILE,
        ControllerCarrello.aggiungiAlCarrello(cliente.getId(), mouse.getId(), 11));
}
```

}

5.3 Casi di test black-box per caso d'uso

Riportiamo per ogni caso d'uso implementato le classi di equivalenza derivate dal flusso degli eventi e i casi di test che ne nascono. In tutto sono 31 casi di test black-box.

Registrare utente (`ControllerUtenti.registra`)

Classi di equivalenza:

Classe	Variabile	Definizione	Validità
CEEV1	email	non ancora registrata	valida
CEENV1	email	già registrata	non valida
CEIV1	immagine	non fornita	valida
CEIV2	immagine	percorso non leggibile	valida
CEIV3	immagine	immagine valida	valida

Casi di test:

ID	Valori (email, immagine)	Classi	Esito atteso
TCBB1	(nuova, assente)	CEEV1, CEIV1	account creato
TCBB2	(già registrata, --)	CEENV1	errore, email già associata
TCBB3	(nuova, non leggibile)	CEEV1, CEIV2	account creato senza immagine
TCBB4	(nuova, valida)	CEEV1, CEIV3	account creato con immagine

Le classi sull'immagine di profilo (CEIV1-CEIV3) sono state ricavate dall'implementazione e non dalla specifica, perché il flusso degli eventi non menziona l'immagine mentre il metodo `registra` la accetta. È l'unico punto del piano in cui il metodo è stato invertito, e va sanato aggiornando la use case description.

Autenticare utente (`ControllerUtenti.autentica`)

Classi di equivalenza:

Classe	Definizione	Validità
CECV1	email registrata e password corretta, ruolo cliente	valida
CECV2	email registrata e password corretta, ruolo amministratore	valida

CECNV1	email non registrata	non valida
CECNV2	email registrata e password errata	non valida

Casi di test:

ID	Valori (email, password)	Classi	Esito atteso
TCBB5	(di un cliente, corretta)	CECV1	accesso, ruolo CLIENTE
TCBB6	(di un admin, corretta)	CECV2	accesso, ruolo AMMINISTRATORE
TCBB7	(non registrata, qualsiasi)	CECNV1	credenziali non valide
TCBB8	(registrata, errata)	CECNV2	credenziali non valide

Le credenziali sono una coppia, quindi le classi non valide sono due, perché sbagliare l'email è diverso dallo sbagliare la password. I due ruoli sono due classi valide distinte, perché dopo l'accesso il sistema smista su funzioni diverse.

[Consultare catalogo prodotti \(ControllerProdotti.getProdotti\)](#)

Classi di equivalenza:

Classe	Definizione	Validità
CECAV1	catalogo privo di prodotti	valida
CECAV2	catalogo con prodotti	valida

Casi di test:

ID	Valori	Classi	Esito atteso
TCBB9	nessun prodotto	CECAV1	elenco vuoto
TCBB10	due prodotti disponibili	CECAV2	elenco con i due prodotti

Essendo un caso d'uso di sola lettura non ci sono classi non valide: l'unica variabile è lo stato del catalogo.

[Aggiungere prodotto al carrello \(ControllerCarrello.aggiungiAlCarrello\)](#)

Classi di equivalenza (prodotto di prova con giacenza 10, intervallo valido 1..10):

Classe	Variabile	Definizione	Validità
CEQNV1	quantità	< 0	non valida
CEQNV2	quantità	= 0	non valida

CEQV1	quantità	1 ... 10	valida
CEQNV3	quantità	> 10	non valida
CEPV1	prodotto	esistente e disponibile	valida
CEPNV1	prodotto	inesistente	non valida
CEPNV2	prodotto	esistente ma non disponibile	non valida

Casi di test:

ID	Valori (quantità, prodotto)	Classi	Esito atteso
TCBB11	(-1, esistente)	CEQNV1, CEPV1	quantità non valida
TCBB12	(0, esistente)	CEQNV2, CEPV1	quantità non valida
TCBB13	(1, esistente)	CEQV1 (limite inf.)	prodotto aggiunto
TCBB14	(5, esistente)	CEQV1 (interno)	prodotto aggiunto
TCBB15	(10, esistente)	CEQV1 (limite sup.)	prodotto aggiunto
TCBB16	(11, esistente)	CEQNV3, CEPV1	non aggiungibile
TCBB17	(1, inesistente)	CEPNV1	prodotto non esistente
TCBB18	(1, non disponibile)	CEPNV2	non aggiungibile
TCBB19	(6 poi 6, esistente)	CEQV1 (accumulo)	non aggiungibile

L'intervallo valido è coperto sui due limiti (1 e 10) più un valore interno (5), con un caso a cavallo di ciascun estremo (0 e 11). TCBB19 non nasce da una classe di equivalenza ma dall'analisi del flusso: il controllo di disponibilità deve tener conto di quanto è già nel carrello, quindi due aggiunte da 6, valide singolarmente, non devono poter superare insieme la giacenza di 10.

Modificare carrello ([ControllerCarrello.modificaQuantita](#))

Classi di equivalenza:

Classe	Variabile	Definizione	Validità
CEMNV1	nuova quantità	< 0	non valida
CEMV1	nuova quantità	= 0 (rimozione)	valida
CEMV2	nuova quantità	1 ... 10	valida
CEMNV2	nuova quantità	> 10	non valida
CERV1	prodotto	presente nel carrello	valida
CERNV1	prodotto	esistente ma non nel carrello	non valida
CERNV2	prodotto	inesistente	non valida

Casi di test:

ID	Valori (quantità, prodotto)	Classi	Esito atteso
TCBB20	(5, in carrello)	CEMV2, CERV1	carrello aggiornato
TCBB21	(0, in carrello)	CEMV1, CERV1	elemento rimosso
TCBB22	(11, in carrello)	CEMNV2, CERV1	quantità non disponibile
TCBB23	(-1, in carrello)	CEMNV1, CERV1	quantità non valida
TCBB24	(2, non in carrello)	CERNV1	non modificabile
TCBB25	(2, inesistente)	CERNV2	non modificabile

Qui lo zero ha un significato proprio: non è un errore ma la richiesta di rimuovere l'elemento, per questo CEMV1 è una classe valida a sé. Letto alla lettera il flusso manderebbe ogni quantità minore o uguale a zero sul ramo di rimozione, incluso -1; il codice invece distingue 0 (rimozione) da negativo (errore), che è la scelta ragionevole. Questo va reso esplicito nel modello, spezzando la condizione nei due rami " $= 0$ " e " < 0 ".

Confermare ordine ([ControllerOrdini.confermaOrdine](#))

Classi di equivalenza:

Classe	Definizione	Validità
CEOV1	carrello non vuoto, tutte le righe disponibili	valida
CEONV1	carrello vuoto	non valida
CEONV2	carrello con almeno una riga non più disponibile	non valida
CEONV3	cliente privo di carrello	non valida

Casi di test:

ID	Valori (carrello)	Classi	Esito atteso
TCBB26	righe tutte disponibili	CEOV1	ordine INSERITO, magazzino scaricato, carrello svuotato
TCBB27	vuoto	CEONV1	ordine non creato
TCBB28	riga divenuta non disponibile	CEONV2	ordine non creato
TCBB29	cliente inesistente	CEONV3	ordine non creato

Non ci sono input da partizionare, l'unica variabile è lo stato del carrello al momento della conferma.

Visualizzare elenco completo ordini ([ControllerOrdini.getOrdini](#))

Classi di equivalenza:

Classe	Definizione	Validità
CEORV1	nessun ordine registrato	valida
CEORV2	almeno un ordine registrato	valida

Casi di test:

ID	Valori	Classi	Esito atteso
TCBB30	nessuno	CEORV1	elenco vuoto
TCBB31	un ordine confermato	CEORV2	elenco con l'ordine e i suoi attributi

Come per il catalogo è un caso d'uso di sola lettura, l'unica variabile è la presenza di ordini.

5.4 Casi di test white-box

Misurata la copertura della suite black-box, il package control non era ancora coperto del tutto: alcuni rami esistono nel codice ma non corrispondono ad alcuna specifica, quindi nessun caso di test derivato dal flusso poteva raggiungerli. Sono soprattutto rami di codice difensivo (per esempio il carrello nullo, che non dovrebbe mai capitare) e il ramo di fallimento del salvataggio. Da qui sono nati cinque casi di test white-box che chiudono i buchi rivelati dalla copertura.

ID	Metodo	Condizione coperta
TCWB1	aggiungiAlCarrello	carrello nullo, prodotto valido
TCWB2	getCarrello	carrello nullo
TCWB3	modificaQuantita	carrello nullo, prodotto valido
TCWB4	registra	percorso immagine di soli spazi
TCWB5	registra	percorso inesistente, solleva IOException

A supporto dei casi di test sui controller ci sono anche i test di unità sulle entità, che verificano il calcolo dei subtotali e del totale, la vendibilità di un prodotto e lo scarico del magazzino. Un dettaglio riguarda `Prodotto.isVendibilePer`, che valuta `disponibile && quantita > 0 && quantita <= quantitaMagazzino`: siccome in Java l'operatore `&&` è in corto circuito, si può arrivare al 100% di copertura di riga e di decisione su quella riga senza aver mai esercitato separatamente le tre condizioni. Per questo ci sono test che rendono falsa una condizione per volta.

5.5 Struttura della suite ed esecuzione

La suite è organizzata su due livelli: test di unità sulle entità e test di integrazione sui controller, dove vivono i casi di test dei casi d'uso. In totale i casi di test del documento sono 36 (31 black-box più 5 white-box) e stanno tutti sui controller, mentre i metodi JUnit complessivi sono

74: i restanti sono i test di unità sulle entità e le verifiche sugli effetti collaterali, che sostengono i casi di test senza esserlo.

I test di integrazione girano su un database H2 in memoria, e non sul MySQL. Il motivo è pratico: una suite che pretende la VPN attiva e il database acceso è una suite che nessuno lancia e che sporca dati condivisi. Il meccanismo è minimo: JpaUtil legge il nome della persistence unit da una system property con default invariato, e maven in fase di test imposta `ecommerce.pu=ecommerceTestPU`, che punta a H2. Il main continua a puntare a MySQL senza alcuna modifica, e `mvn test` gira senza VPN e senza database acceso.

```
private static final String NOME_PU =  
    System.getProperty("ecommerce.pu", "ecommercePU");
```

5.6 Risultati di copertura

Il dato complessivo è del 39% perché l'interfaccia grafica non è testata, e da sola pesa 436 righe su 802. La verifica è stata concentrata sulla logica di business (entity e control), separandola dalla presentazione come indicato dagli appunti. Il 100% su entity e il 96% su control descrivono quindi la copertura degli strati che contengono la logica applicativa.

Resta un ramo strutturalmente scoperto, il fallimento del salvataggio: non è raggiungibile con un database vero, servirebbe un mock e quindi introdurre la dependency injection nei controller. Abbiamo scelto di non riscrivere l'architettura per un singolo ramo, coerentemente con la formula "100% di line e branch coverage, ove possibile".

5.7 Cosa ha trovato la suite

La progettazione dei casi di test a partire dalle specifiche ha fatto emergere due difetti, poi corretti.

Il primo: `isVendibilePer` controllava solo il limite superiore, quindi una quantità negativa passava il controllo. Il problema si propagava fino alla conferma dell'ordine, dove `scarica(-1)` esegue `quantitaMagazzino -= -1`: confermare un ordine con quantità negativa aumentava la giacenza a magazzino. È stato corretto alla radice aggiungendo `quantita > 0` a `isVendibilePer`.

Il secondo: dove `aggiungiAlCarrello` restituiva un codice d'errore pulito su prodotto inesistente, `modificaQuantita` sollevava una `NullPointerException`. È stato corretto allineando i controlli sui riferimenti nulli. Da notare che il primo test scritto per questo caso passava, dando un falso senso di sicurezza: con il carrello vuoto il ciclo interno non parte e l'eccezione non scatta. Il bug è emerso solo rafforzando il test con un carrello popolato, a conferma che un test verde non significa che il caso sia coperto, ma solo che quel test è passato.

5.8 Tracciabilità test / casi d'uso / requisiti

La tabella lega i casi di test ai casi d'uso e ai requisiti verificati, per controllare la copertura delle funzionalità principali del sistema.

Caso d'uso	Requisiti	Casi di test
Registrare utente	RF01, RN03	TCBB1-TCBB4, TCWB4, TCWB5
Autenticare utente	RF01, RN03	TCBB5-TCBB8
Consultare catalogo prodotti	RF03	TCBB9-TCBB10
Aggiungere prodotto al carrello	RF04, RD01	TCBB11-TCBB19, TCWB1
Visualizzare carrello	RF04	TCWB2
Modificare carrello	RF04	TCBB20-TCBB25, TCWB3
Confermare ordine	RF05, RD02	TCBB26-TCBB29
Visualizzare elenco completo ordini	RF07	TCBB30-TCBB31

6. Manuale utente e istruzioni di installazione

6.1 Repository

Il codice sorgente del progetto è disponibile nella repository:

https://github.com/raffaeleedidonna/Gruppo_IS_31

6.2 Requisiti per l'esecuzione

Per compilare ed eseguire il sistema servono:

- JDK 21 o superiore; - Apache Maven; - accesso al database MySQL del progetto, raggiungibile all'indirizzo 192.168.178.103:3306 tramite la rete privata WireGuard. Senza la VPN attiva l'applicazione non riesce a connettersi al database.

La configurazione della connessione è nel file

src/main/resources/META-INF/persistence.xml, nella persistence unit ecommercePU, dove sono indicati indirizzo, nome del database, utente e password. Chi volesse usare un database diverso deve modificare quei parametri.

6.3 Installazione ed esecuzione

Dopo aver clonato la repository, spostarsi nella cartella JavaProject che contiene il progetto Maven. Da lì:

- avviare l'applicazione con interfaccia grafica da IDE o da terminale con:

```
cd JavaProject
mvn compile
```

```
mvn exec:java
```

- per eseguire la suite di test, che non richiede né VPN né MySQL perché gira su database H2 in memoria:

```
cd JavaProject  
mvn test
```

Al primo avvio con MySQL, Hibernate crea da solo le tabelle mancanti, perché la persistence unit è configurata con `hibernate.hbm2ddl.auto=update`.

6.4 Principali funzionalità e modalità d'uso

All'avvio l'applicazione mostra la schermata di accesso. Da qui un visitatore può registrarsi, compilando il modulo con i propri dati, oppure autenticarsi con email e password. Le password non vengono mai memorizzate in chiaro, ma salvate come hash.

Dopo l'accesso, l'utente arriva alla schermata principale, da cui può raggiungere le varie sezioni. La schermata si adatta al ruolo: il cliente vede anche il pulsante del carrello, l'amministratore no.

Un **cliente** può:

- consultare il catalogo dei prodotti, con nome, descrizione, prezzo, disponibilità, categoria ed eventuale offerta;
- aggiungere un prodotto al carrello indicando la quantità, con il sistema che impedisce di superare le scorte disponibili;
- visualizzare il carrello con i subtotali e il totale complessivo, modificare la quantità di una riga oppure rimuoverla portandola a zero;
- confermare l'ordine, dopo di che riceve una notifica di conferma e il carrello viene svuotato;
- consultare lo storico dei propri ordini con data, totale e stato.

Un **amministratore** può visualizzare l'elenco completo di tutti gli ordini registrati nel sistema.

Le notifiche, nella versione attuale, vengono scritte sul log dell'applicazione tramite un notificatore dedicato. L'architettura è predisposta per sostituire questo canale con altri (per esempio l'email) senza modificare la logica applicativa.